



CG2111A Engineering Principle and Practice
Semester 2 2024/2025

“Alex to the Rescue”
Final Report
Team: B05-08-4B

Name	Student #	Main Role
Chong Ruolin	A0302910W	Overall designer and engineer, colour detection algorithm
Manikanda Durairaj Prathistta	A0282337B	State-of-the-art researcher, wiring expert
Zhu Wenbo	A0307272J	Teleoperation programmer, overall firmware designer
Zhu Yicheng	A0309066A	TLS and SLAM programmer, Alexandra’s personal driver

TABLE OF CONTENTS

1. Introduction	4
2. Review of State of the Art	4
2.1. Earthshaker	4
2.2. Boston Dynamics Spot	5
3. System Architecture	5
4. Hardware Design	6
4.1. Pictures of the Alex	6
4.2. Alex Components and Their Functions	6
4.3. Additional Hardware Features	7
4.3.1. Wire management	7
4.3.2. Weight distribution	7
4.3.3. Ice cream sticks	7
5. Firmware Design	8
5.1. High-Level Algorithm on Arduino	8
5.2. Communication Protocol	8
5.3. Bare Metal Programming	8
5.3.1. Servo	8
5.3.2. UART	9
6. Software Design	9
6.1. High-Level Algorithm on Pi	9
6.1.1. Teleoperation	10
6.1.2. Colour Detection	12
6.2. Moving SLAM to PC	12
6.2.1. Communication with TLS	13
6.2.2. LiDAR Scanning on RPi	13
6.2.3. SLAM processing on PC	13
6.3. Additional Software Features	14
6.3.1. Non-blocking Key Input for Smoother Control	14
6.3.2. Fancy Alex Control Panel	14
6.3.3. LiDAR Scans Filtering	14
7. Conclusions	14
7.1. Lessons Learnt	14
7.2. Mistakes Made	15
8. References	16
9. Appendix	17
9.1. Bare Metal Servo Code	17
9.2. Bare Metal UART Code	17
9.3. Main Teleoperation Logic	20
9.4. Movement Control based on Time Duration	20
9.5. findColor() Code	23
9.6. avgFREQ() Code	24
9.7. getColor() Code	24

9.8. Nodes enabled on RPi	25
9.9. Nodes enabled on PC	25
9.10 getch() Code	25
9.11. Fancy Alex Control Panel	26
9.12. Filtering Algorithm for Lidar Scans	27

1. Introduction

The “Search and Rescue” problem Alex is designed to tackle requires it to navigate an unknown environment using the LIDAR data and “rescue” the astronauts, with each astronaut requiring different attention based on their colour.

In order to find the astronauts, Alex will minimally need to be able to be remotely controlled to move straight both forwards and backwards and be able to turn at customisable angles to go around bends and have its LiDAR placed at most 21 cm high to be able to detect the obstacles and astronauts. When a semicircle arc is detected on the LiDAR, the camera can be used to confirm that the object is an astronaut.

After the astronaut is identified, the colour sensor will be used to identify the astronaut, and the medpak will be dropped if it is green, or the astronaut will be dragged to safety if it is red.

Throughout the operation, Alex will also map out its surroundings to find the most efficient way of bringing the red astronaut to safety, given the time constraints.

2. Review of State of the Art

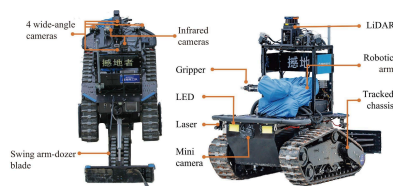


Figure 2.1

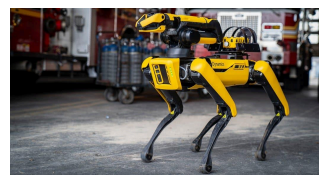


Figure 2.2

2.1. Earthshaker

Earthshaker is a rescue robot designed for disaster, integrating a tracked chassis with a swing arm for dexterous manipulation and a dozer blade for obstacle clearing. With 24 channels to transmit commands, various aspects of the robot can be controlled simultaneously.

Strengths:

1. **High Redundancy:** The system primarily uses a 2.4GHz frequency communication system, which allows for fast and low-latency control under short-range communication. The 1.8 GHz MIMO-mesh radio and 4G/5G network-based telecommunications link serve as a backup when the 2.4 GHz signal degrades due to increased distance/signal interference from obstacles.
2. **Comprehensive sensor array:** 4 wide-angle cameras, 2 infrared cameras, a mini camera, a depth camera, and LiDAR enable 360° situational awareness and detailed obstacle detection. Alex uses just LiDAR and a monochrome Pi camera - enough for flat-ground navigation without unnecessary processing load.

Weaknesses:

1. **Limited lifting:** The robotic arm and gripper, optimised for dexterous tasks, lack the capacity to safely lift and reposition heavier objects, as it has a payload of only 5 kg.
2. **Short battery life (3 hours):** High energy demands from 1000W motors, heavy chassis (~250Kg) and numerous sensors limit operation of the system under a

moderate workload. Alex, in contrast, has a comparable 3-hour battery life while consuming only 9.6W.

2.2. Boston Dynamics Spot

Spot is an agile quadruped robot used in rescue operations to relay critical data to support rescue teams and assist victims. It offers a large range of motion and maintains balance.

Strengths:

1. **High customisation:** Spot can be outfitted with various payloads, making it adaptable for many applications. Comprehensive documentation and a robust API allow users to equip the robot for their specific use cases.
2. **Secure data transmission:** Leverages military-grade data encryption using AES-XTS 256-bit encryption for onboard storage, with keys bound to its Trusted Platform Module and validated by UEFI Secure Boot. It enforces TLS 1.0 or later for all web communications, protecting critical data. Digitally signed software updates and third-party security audits keep it resilient to cyber threats in hostile environments. Alex also implements TLS encryption, transmitting raw LiDAR data from the RPi to a PC via a custom TLS socket.

Weaknesses:

1. **Not suitable for all terrains:** Unable to walk on soft or pliable surfaces as it needs a minimum coefficient of friction of 0.4 to maintain balance.
2. **Unable to operate in narrow areas:** Requires at least 60 cm of clearance for corridors and 70 cm of overhead space. Alex, however, only needs a clearance and overhead of 17 cm.

3. System Architecture

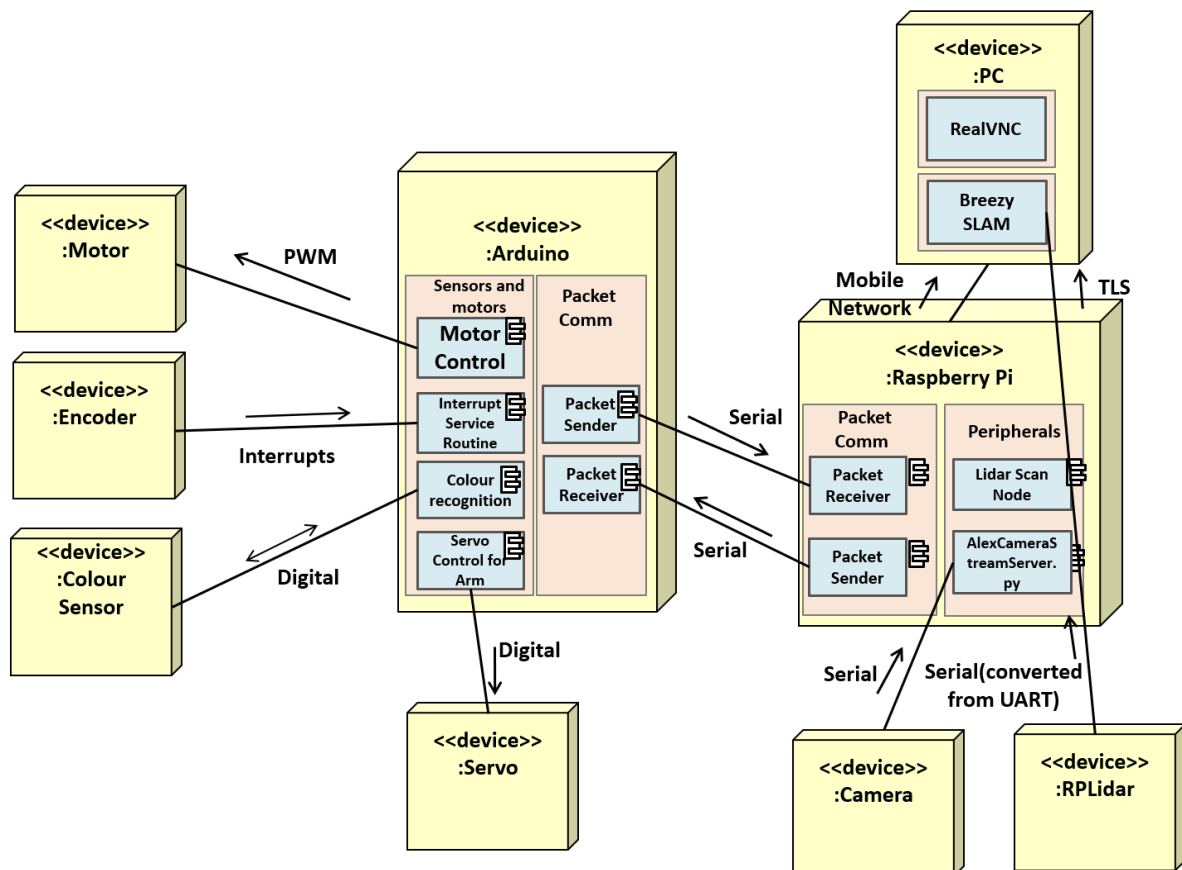
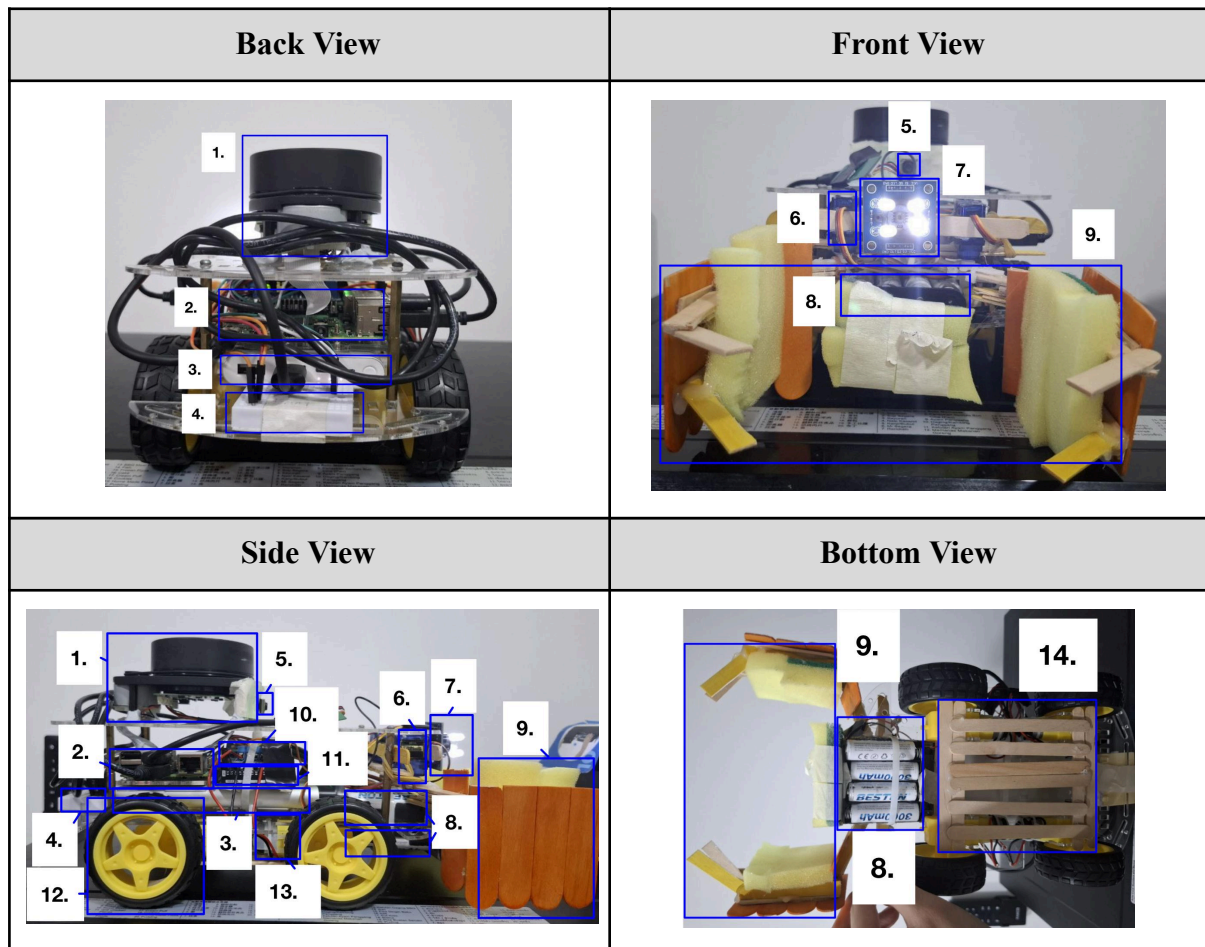


Figure 3.1: System Architecture UML

4. Hardware Design

4.1. Pictures of Alex



4.2. Alex Components and Their Functions

No.	Component	Description
1	LiDAR	LiDAR performs 360° scans to generate the map. Placed on the top of the robot to prevent other components from blocking it during scanning.
2	Raspberry Pi	Controls the robot upon receiving commands from the driver. Placed at the back and on top of the power bank for easy power supply.
3	Powerbank	Used to power the RPi.
4	Breadboard	Used to connect the 5V and GND pins from the Arduino to the colour sensor and servo motors.
5	Camera	Used at specific times during the run to identify astronauts.
6	Servo motor	Used to control the opening and closing of the claw.

7	Colour sensor	Used to identify the colour of the astronaut
8	Battery holder	Soldered 2 4-battery holders together to power the motor driver.
9	Claw	Used to transport the medpak to the green astronaut and transport the red astronaut to safety.
10	Motor shield	Used to connect the motors to the Arduino, mounted on the Arduino.
11	Arduino	Used to control the servo motors, motors and colour sensors, gets code uploaded from the RPi.
12	Wheels	Turns to move the robot around.
13	Motor	Used to control directions and speed of the wheels, based on code uploaded to Arduino for moving forward, backwards and turning.
14	Ice cream sticks	Used to ensure that the structure does not cave in since the robot is rather heavy.

4.3. Additional Hardware Features

4.3.1. Wire management

Although marks are not deducted for touching the wall, we found that sometimes the wires would catch onto the walls, resulting in the robot being unable to move. Hence, to prevent this, we used tape and zip ties to make sure that the wires do not stick out of the robot when possible.

4.3.2. Weight distribution

As Alex became heavier with the additional components added, we realised that the manoeuvring became slightly harder, as we were unable to turn at times. Hence, taking into account the weight distribution, we tried to make sure that the heavier components, like the power bank, were in the lower level and closer to the centre of the robot. This made the centre of gravity closer to the middle, which aided Alex's turning.

4.3.3. Ice cream sticks

As we added all the components, we found that the weight was putting pressure on the motors, so to prevent the robot from caving in, we added extra ice cream sticks to ensure Alex's motors remain level with each other, allowing it to move smoothly.

5. Firmware Design

5.1. High-Level Algorithm on Arduino

While serial communication between the Arduino and Raspberry Pi is active, the Arduino continuously polls for incoming data packets and performs the necessary checks as follows:

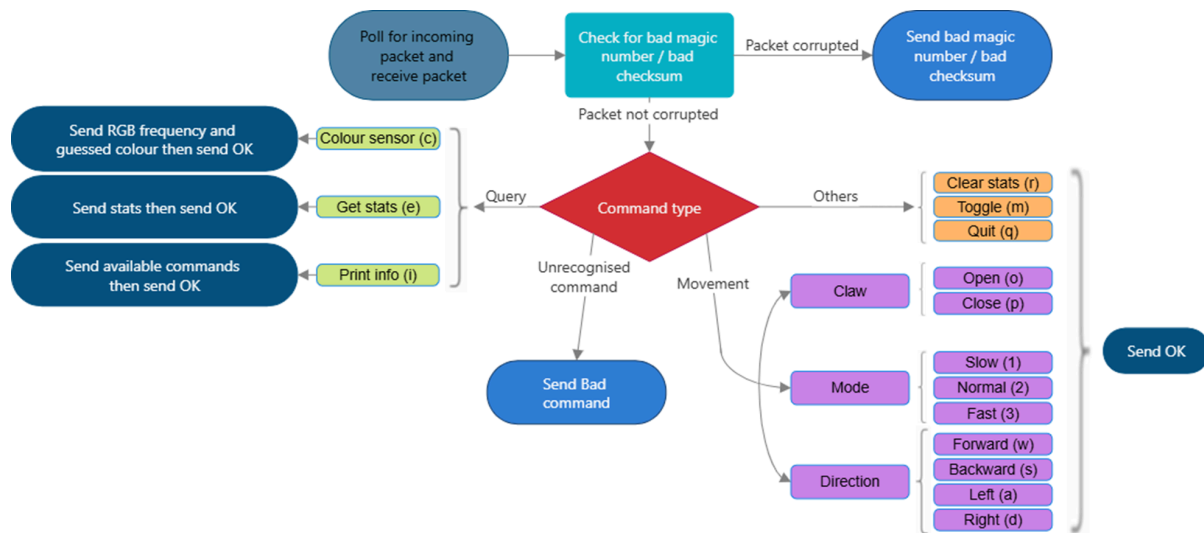


Figure 5.1: High-Level Algorithm on Arduino

5.2. Communication Protocol

The Arduino and RPi communicate asynchronously via UART with the standard **8N1** frame format with a baud rate of 9600 bps. After accounting for the magic number, checksum and the padding bytes, each packet is 140 bytes long. This means that the time taken for RPi to send a command to Arduino and vice versa is $(140 \times 10) \div 9600 \approx 0.146\text{s}$ or 146ms. So, if the driver sends commands too fast, let's say sending commands at a 100ms interval, when deserialising, the result will be **PACKET_BAD**, which, when handled, will print "ERROR: Bad Magic Number". We've solved this issue in [6.1.1.1](#). After the Arduino completes executing a command, it sends a response packet to the Pi, and RPi will handle the response accordingly.

5.3. Bare Metal Programming

5.3.1. Servo

The biggest challenge for converting our code (See [Appendix 9.1](#)) to bare metal was identifying the correct OCR5X values to emulate our arm opening and closing. Originally, our arm opens with the left and right servos at 0° and 90° respectively and closes at 45°.

Servo Angle	Pulse Width	OCR5A/B
0°	1.0 ms	2000
45°	1.25ms	2500
90°	1.5ms	3000

Table 5.3.1: Servo values converted

Based on the convention for servo motors¹, we found the pulse width for the above servo angles. Now we need to generate the specific pulse width in our bare metal code.

¹ In the [servo motor SG90 datasheet](#), it is actually stated that 1.5ms is the middle (0°), 2.0ms = 90°, 1.0ms = 90°. However, based on our own tests, we find that 1.0ms and 2.0ms match 0° and 90° respectively for Arduino code. So either the angle in the datasheet is different from the degrees in the Arduino servo library, or there are differences due to different manufacturers.

First, we calculate the duration for each timer tick. By using a prescaler of 8, we get that each timer tick is $(\frac{16,000,000}{8})^{-1} = 0.5\mu\text{s}$. Since SG90 servos typically have a frequency of 50Hz, one PWM period is 20ms. So we need $\frac{20,000}{0.5} = 40,000$ ticks for one PWM period, which fits in our 16-bit timer, setting $\text{ICR5} = 40,000$. The rest is just a matter of calculating the OCR5A/B value (how many ticks are required for the pulse width) for each angle we need the servos to go to.

5.3.2. UART

We also updated our code for UART communication between Arduino and RPi to bare metal (See [Appendix 9.2.](#)). However, a problem we ran into was compatibility issues with the `serialize` Arduino library that was provided to us during labs. The first issue was with `serialize.cpp`, where the `stdio.h` library is not compatible with bare metal AVR and was removed. Any mentions of `printf` also had to be removed.

The second issue was with `serialize.h`, where the file started with `#ifndef __SERIALIZE__ #define SERIALIZE`. This is actually a typo from the original file where the `define` does not match the `ifndef`, meaning `__SERIALIZE__` was not defined in the guard. This means that the compiler will not realise that the contents of `serialize.h` are included more than once, causing a redefinition error. After some research, this error likely only occurs for bare metal, but not before, because bare metal allows the files to be compiled separately instead of merging all the `.ino` files together, causing the conflict to happen.

6. Software Design

6.1. High-Level Algorithm on Pi

1. Initialise communication between PC, RPi and Arduino, and run the LiDAR and Camera
2. Generate the map using SLAM on PC, update with live scans from RPi connected to PC over TLS connection
3. RPi deals with sending commands and receiving responses using two threads
 - a. Sending Command (on `main` thread): The Driver sends the command from RPi to Arduino based on the data
 - b. Receiving Response (on `recv` thread): RPi handles the response packet from Arduino
4. Arduino processes the command and sends the response packet to RPi
5. Repeat steps 2-4 until Alex reaches the parking space

6.1.1. Teleoperation

6.1.1.1. Teleoperation Logic on RPi

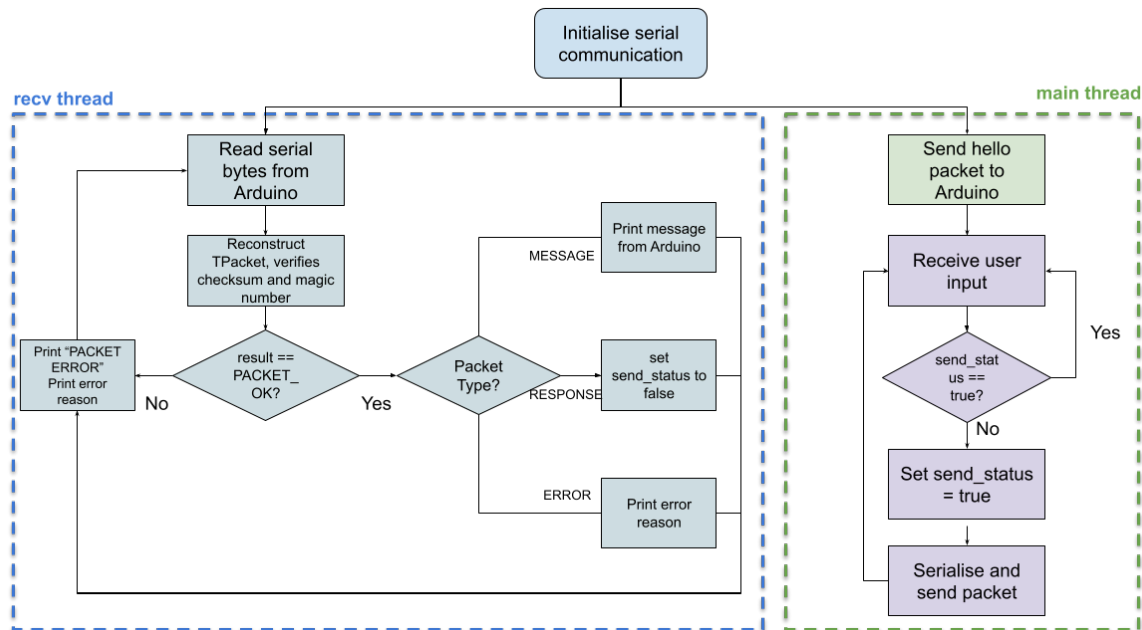


Chart 6.1.1.1: Teleoperation Logic on RPi

In the Alex control executable **Alex-pi** (Compiled via **gcc** command), there are two threads: 1) the **main** thread, used to send commands from RPi to Arduino, and 2) the **recv** thread, used to receive responses from Arduino to RPi. These two threads run concurrently. One thing to note is that we add a **send_status** flag to indicate whether the command should be sent to Arduino or not. This flag is set to **false** once RPi receives a response packet with **RESP_OK**, meaning that we can send the new command. Otherwise, nothing will be sent (See [Appendix 9.3](#)). This mechanism ensures that the system does not overwhelm the Arduino with rapid commands and effectively prevents "Bad Magic Number" errors, even when the driver continuously presses input keys.

6.1.1.2. Alex Control Commands

To enhance the smoothness and intuitiveness of controlling Alex, we have remapped the movement controls to the standard WASD key configuration. There are a total of 15 commands (highlighted in **dark red**), which are categorised into the following functional groups based on their respective roles.

6.1.1.2.1. Movement commands:

Commands (WASD + Claw Control)			
Forward	Backward	Left	Right
W	A	S	D
Open Claw		Close Claw	

O	P
----------	----------

Table 6.1.1.2.1a: Commands (WASD)

Speed Mode				Manual Mode
<i>Mode</i>	<u>Slow</u>	<u>Normal</u>	<u>Fast</u>	Requires direction, movement duration as input. This mode is used as a backup in case the three speed modes are not enough to explore the moon.
<i>Command</i>	1	2	3	
<i>Preset Movement Duration</i>	0.05s	0.1s	0.25s	
<i>Description</i>	Used for operations that need precise control, like grabbing the astronaut.	Used for exploring the moon at a safe speed.	Used for the faster delivery of the medpak and the rescue of the astronaut.	

Table 6.1.1.2.1b: Speed Mode and Manual Mode

Alex can operate in two modes: 1) Speed Mode and 2) Manual Mode. By default, Alex is configured to run in Speed Mode. However, the driver can toggle between the two modes by pressing the ‘m’ key. In Speed Mode, we use preset durations to determine the distance travelled and the angle turned (See [Appendix 9.4](#)). This approach provides greater stability compared to using encoder-based distance and angle measurements.

6.1.1.2.2. Query commands:

Command	Key	Description
Get Colour Data	C	Sends the RGB values and the predicted colour from the colour sensor to the Raspberry Pi via a response packet, and displays the information on the RPi terminal.
Get Odometry Data	E	Sends the encoder status to the Raspberry Pi via a response packet, and displays the information on the RPi terminal.
Get available commands	I	Directly calls the method to display the list of available commands on the RPi terminal.

Table 6.1.1.2.2: Query Commands and Their Descriptions

6.1.1.2.3. Other valid commands:

Command	Key	Description
Clear Odometry Data	R	Resets all variables storing the encoder status to zero.
Toggle Mode	M	Toggles between Speed Mode and Manual Mode.
End Session	Q	Terminates the serial communication session between the RPi and the Arduino.

Table 6.1.1.2.3: Other Commands and Their Descriptions

6.1.2. Colour Detection

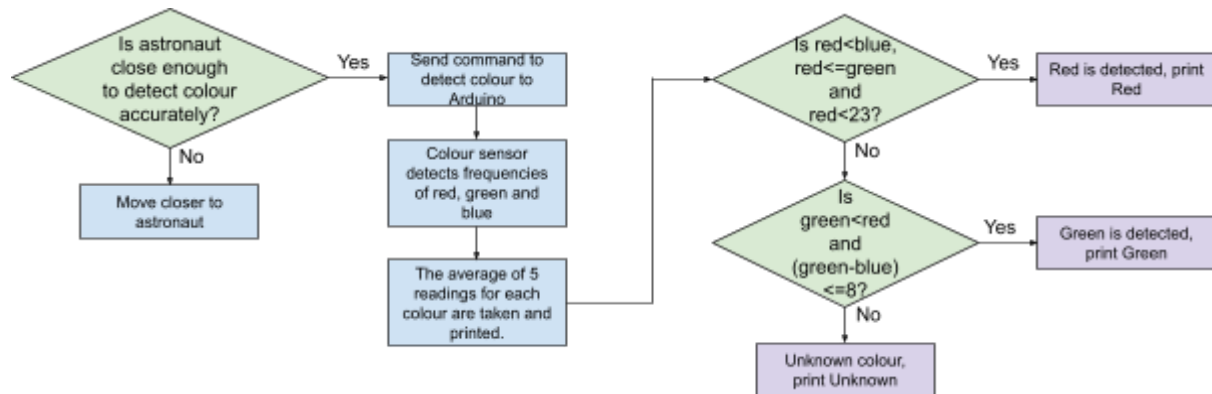


Chart 6.1.2: Colour detection algorithm

When the astronaut is close enough to Alex based on the LiDAR scan, the command to detect colour is sent to the Arduino by pressing C, and the `findColor()` method is called (See [Appendix 9.5.](#)), which calls the `avgFREQ()` method for the 3 colours, red, green and blue. The `avgFREQ()` method takes the average frequency over 5 readings for more reliable results (See [Appendix 9.6.](#)). The red, green and blue frequency variables are then updated to hold the new values. The `getColor()` method is then called (See [Appendix 9.7.](#)), where the detected colour, red, green, or unknown, is determined using the above algorithm. The `detectedColor` variable is then updated to hold the new colour. This detected colour, together with the red, green, and blue frequencies, is then sent using the packet system to the RPi. The RPi then handles the received packet, printing the red, green and blue frequencies, together with the guessed colour. During the testing, our group realised that the predicted colour was not always accurate. Hence, instead of fine-tuning the threshold values, which may change due to differences in lighting, we decided to determine the colour using the red and green frequencies printed, based on whichever colour's frequency was the lowest.

6.2. Moving SLAM to PC

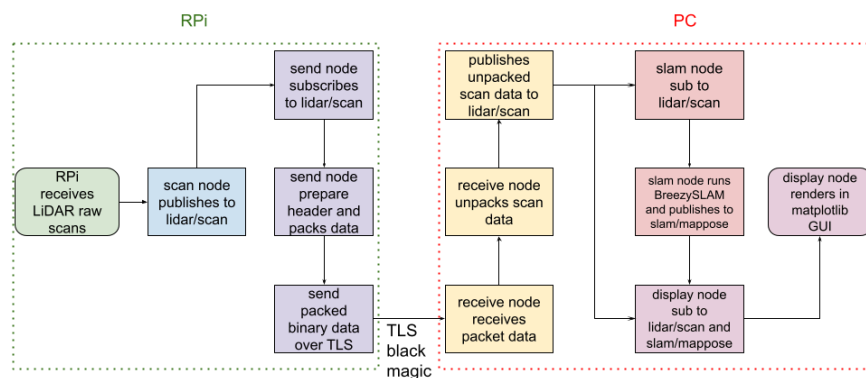


Chart 6.2: SLAM algorithm

While the Raspberry Pi we were provided with is a capable little computer, it is limited by its processing power and RAM, which is feeble compared to our Intel or AMD chipsets within our computers. Hence, despite the SLAM code provided to us being able to run out of the box on the RPi, our group took a step further to move the SLAM computation and mapping

processes onto our laptops. By making use of the pub/sub manager that was provided to us, we were able to cleanly and easily (a lie, it was anything but easy) offload the SLAM and plotting processes to our PCs, which massively improved the framerate of the scan plot and SLAM map.

6.2.1. Communication with TLS

Before our data is able to be transported through the TLS socket, we need to be able to set up a TLS server for our RPi to communicate with our PC. So we created the necessary certificates and keys, storing the server key and cert on the PC and client key and cert on the RPi. We also had to set up port forwarding on our Windows PC to forward any communication with our Windows IP address to our Ubuntu IP address.

Most importantly, we have to make modifications as the TLS nodes that were provided to us were meant to be used for communication with the Arduino. We added a new packet type that we called `NET_LIDAR_PACKET` and assigned 4 to it to be used as the header byte. This way, the server can recognise when it receives the LiDAR scans and unpack it back to the format of a tuple of three lists, angle, distance, and quality.

6.2.2. LiDAR Scanning on RPi

The pub/sub system provided to us allowed us to easily disable every irrelevant node on the RPi. In fact, we only have to preserve two nodes, the LiDAR node and the TLS send node, as these will be the only job of the RPi now, taking in the raw data from the LiDAR scans and sending them over TLS to our PC. We also opened up TCP port 5000 in Windows Firewall.

Now, this is where the fun begins. While the LiDAR node still publishes to the `lidar/scan` topic like the original code, we have the TLS send node subscribe to the topic now instead of the SLAM node. The TLS send node will encode the raw scans into bytes, save them in a byte array and send it over a TLS socket to the PC. (See [Appendix 9.8.](#))

6.2.3. SLAM processing on PC

On the PC side, the same libraries used on the RPi were installed, and the virtual environment was set up. However, as we are offloading the SLAM and plotting process, we need to enable the SLAM, display and TLS receive nodes. We can disable the LiDAR nodes since we already have the raw scans sent to us.

Before the transmission begins, the PC has already started the TLS server so that the RPi can connect to the PC over TLS. As the TLS server receives the data, the TLS receive node identifies the data type that was sent over with the header byte. Once it confirms the data packet is indeed the LiDAR scans, it unpacks the byte array back into the (angle, distance, quality) tuple and publishes the unpacked raw data to the `lidar/scan` topic on PC's pub/sub system. (See [Appendix 9.9.](#))

The rest of the process is exactly the same as what happens on the RPi in the original code provided. The SLAM node and display node both subscribe to the `lidar/scan` topic. The

SLAM node will process the scans using BreezySLAM and publish the SLAM map data to `slam/mappoint`. The display node is subscribed to both `lidar/scan` and `slam/mappoint`, where it updates the left side of the screen with the live scan points from `lidar/scan` and plots the SLAM map on the right side of the screen with the data from `slam/mappoint`.

6.3. Additional Software Features

6.3.1. Non-blocking Key Input for Smoother Control

In the original control template for Alex, the driver was required to press the Enter key after each command input. This approach, known as blocking key input, introduces a noticeable delay and can hinder the smoothness of the driving experience. To address this, we researched alternative methods and found a solution on Stack Overflow (Patel, 2011). Based on that, we implemented non-blocking key input for Alex's Speed Mode, allowing real-time command execution without requiring the Enter key (See [Appendix 9.10.](#)). Meanwhile, we retained blocking input for Manual Mode, where precision and deliberate control are prioritised.

6.3.2. Fancy Alex Control Panel

To enhance the driver's driving experience, we improved our Alex Control Panel to be fancier and more intuitive. (See [Appendix 9.11.](#))

6.3.3. LiDAR Scans Filtering

To improve the performance of the LiDAR and reduce noise, i.e. scan points detected that are not actual objects, we decided to filter out any points that seem, to quote the youth nowadays, "sus". When we read the raw values directly, we receive a tuple of 3 lists. By consulting the [datasheet](#) for the RPLIDAR we are using, we can tell that the lists are of the angle, distance and quality, in degrees, mm and level, respectively. Hence, we filtered out any data with a distance under 100mm and quality under 15 (from the raw value, there are only scans with qualities of either 0 or 188). (See [Appendix 9.12.](#))

7. Conclusions

7.1. Lessons Learnt

The first important lesson our group learnt was to improve our productivity. During this project, we found ourselves racing against time to create a fully functioning robot by the time the demo and final run came. This was especially so due to the complexity of our robot, which needed to have various features such as the motors, the communication between our computers and the Raspberry Pi, the colour sensor and the claw, etc. Since some of these parts could be coded individually before being pieced together, our group decided to divide the workload. For example, a group member could work on the controls while another could work on the colour sensor before piecing everything together. However, since there was only one robot, the group was not able to test their code out individually. This was where the GitHub platform came into play, allowing our group to collaborate on the code online,

allowing the person who had the bot to test out the code written by the other groupmates, working around the issue of only having one robot.

The other important lesson our group learnt was to streamline our setup processes. This was done by creating a script that could be run instead of typing out every command every time we wanted to set up the robot. This was especially important for our group since we tried to conserve battery power by turning off the robot whenever it was not running to allow us to maximise our testing time in the lab. By creating the script, we did not need to go through the setting up of the virtual environment every single time we restarted the robot. This saved us a lot of valuable time, not only when we were testing the robot in the lab, but also during the 5 minutes of set-up time we were given before the final run. As a result, we were able to better allocate our limited time towards improving and perfecting our robot, ensuring that it met all the demands of the project, given the time constraints.

7.2. Mistakes Made

One mistake we made was not ensuring that our circuit was debugging-friendly from the start. Initially, we secured the wires with tape to prevent them from sticking out; however, what we did not foresee was that it would make it difficult for us to trace the connections. Hence, when one component, say the servo motor, was not working, we were unable to tell at a glance if the problem was hardware related, like a wire coming loose. In the initial stages of testing, when our servo motor did not work as expected, we took apart the entire robot to check the wirings, but in the end found that it was not a wiring issue and the robot worked after we turned the Raspberry Pi off and then back on, leading us to deduce that it was either a temporary software glitch or an initialisation issue. In the last few days leading up to our final run, we ran into the same issue and this time, turning on and off the Raspberry Pi was no longer solving our problems. Since by then, our robot was already finalised and we had secured it with zip ties and hot glue, we thought that taking apart the robot and redoing all the wiring would be too time consuming, and hence opted for a temporary fix, which was to just tape the wires down and hope that the connections stayed in place. While it did work briefly, the problem came back, and that was when we decided to take apart the whole robot and change any wires that were even slightly loose, which admittedly took up quite a lot of time. In hindsight, if we had checked for loose connections earlier and made our wiring less messy, this issue could have been completely avoided.

Another mistake our group made was not keeping track of the updated files. When running the robot, we thought that a constant used was 100 when it was actually 80. This led to the robot not being able to turn or move properly since it was too heavy to run at the lower value. However, we thought that the code we were running was already at the maximum, and hence, we turned our efforts to debug the hardware aspect, such as making sure the batteries were fully charged, and even considered changing all the motors. In the end, when all else failed, we went back to look at the code, tracing the issue back to the incorrect constant in the code. This could have been avoided if we had maintained better version control, with each file's name telling us what adjustments had been made instead of just calling the files "Alex", for example.

8. References

Boston Dynamics Support Center. (2024b). Bostondynamics.com.

<https://support.bostondynamics.com/s/article/About-the-Spot-Robot-72005>

Boston Dynamics Support Center. (2024). Bostondynamics.com.

<https://support.bostondynamics.com/s/article/Nominal-Operating-Environments-for-Spot-49930>

Boston Dynamics. (2023). *Spot*. Boston Dynamics.

<https://bostondynamics.com/products/spot/>

Zhang, Y., Li, Y., Zhang, H., Wang, Y., Wang, W., Ye, Y., Yue, Y., Guo, N., Gao, W., Chen, H., & Zhang, S. (2023). Earthshaker: A mobile rescue robot for emergencies and disasters through teleoperation and autonomous navigation. *Journal of University of Science and Technology of China*, 53(1), 4–4.

<https://doi.org/10.52396/justc-2022-0066>

Patel, J. (2011, September 19). *What is the equivalent to getch() & getche() in Linux?* Stack Overflow.

<https://stackoverflow.com/questions/7469139/what-is-the-equivalent-to-getch-getche-in-linux>

Imperial College London, Servo Motor SG90 datasheet

http://www.ee.ic.ac.uk/pcheung/teaching/DE1_EE/stores/sg90_datasheet.pdf

9. Appendix

Full code: <https://github.com/mendax1234/CG2111A-Final-Project>. All .cpp and .ino files are formatted using `clang-format` following the [CS1010 coding style guidelines](#).

9.1. Bare Metal Servo Code

```
void setupArm() {
    // Pin 45 and 46 are used instead due to lack of PWM on pin 15
    and 16
    DDRL |= 0b00011000;

    // Non-inverting PWM on OC5A and OC5B
    TCCR5A = 0b10100010;
    // Fast PWM, prescaler = 8
    TCCR5B = 0b00011010;

    ICR5 = 40000;
    OCR5A = 2000;
    OCR5B = 3000;
}

void openArm() {
    OCR5A = 2000;
    OCR5B = 3000;
}

void closeArm() {
    OCR5A = 2500;
    OCR5B = 2500;
}
```

9.2. Bare Metal UART Code

```
/*
 * Setup and start codes for serial communications
 * Remove stdio.h and printf in serialize.cpp for compatibility
 * Ensure headers for serialize.h are: #ifndef __SERIALIZE__
```

```

* #define __SERIALIZE__
*/
#include "packet.h"
#include "serialize.h"
#include <avr/io.h>

#ifndef F_CPU
#define F_CPU 16000000UL
#endif
#define BAUD 9600
#define UBRR_VALUE ((F_CPU / (16UL * BAUD)) - 1)

#define MAX_BUFFER_LEN 1024

void setupSerial() {
    UBRR0H = (unsigned char)(UBRR_VALUE >> 8);
    UBRR0L = (unsigned char)(UBRR_VALUE);

    UCSRB = 0b00011000; // RXEN0 and TXEN0
    UCSRC = 0b00000110; // UCSZ01 and UCSZ00 for 8-bit, no parity
}

void startSerial() {
    // setupSerial already start serial connection
}

// Read the serial port. Returns the read character in
// ch if available. Also returns TRUE if ch is valid.
// This will be replaced later with bare-metal code.

int readSerial(char *buffer) {
    int count = 0;

    while (UCSR0A & 0b10000000) { // RXC0: Receive Complete
        buffer[count++] = UDR0;
        if (count >= MAX_BUFFER_LEN)

```

```

        break;
    }
    return count;
}

void writeSerial(const char *buffer, int len) {
    for (int i = 0; i < len; ++i) {
        while (!(UCSR0A & 0b00100000))
            ; // Wait until UDRE0: Data Register Empty
        UDR0 = buffer[i];
    }
}

/*
 *
 * Alex Communication Routines.
 *
 */
TResult readPacket(TPacket *packet) {
    // Reads in data from the serial port and
    // deserializes it. Returns deserialized
    // data in "packet".

    char buffer[PACKET_SIZE];
    int len;

    len = readSerial(buffer);

    if (len == 0)
        return PACKET_INCOMPLETE;
    else
        return deserialize(buffer, len, packet);
}

void sendResponse(TPacket *packet) {
    // Takes a packet, serializes it then sends it out

```

```

// over the serial port.
char buffer[PACKET_SIZE];
int len;

len = serialize(buffer, packet, sizeof(TPacket));
writeSerial(buffer, len);
}

```

9.3. Main Teleoperation Logic

```

while (!exitFlag) {
    char ch;
    if (printInst) {
        printInstructions();
        printInst = false;
    }
    if (manual) {
        printf("Enter command: ");
        scanf(" %c", &ch);
        flushInput(); // Purge extraneous characters from input stream
    } else {
        ch = getch(); // Auto mode
    }

    manual = (ch == MANUAL) ? !manual : manual; // Toggle only when
    'm' clicked

    if (!send_status) {
        send_status = true;
        sendCommand(ch, manual, &currentMode);
    }
}

```

9.4. Movement Control based on Time Duration

Motor.ino Code

```

// Direction control functions for left motors

```

```

void leftMotorsForward() {
    motorFL.run(FORWARD);
    motorBL.run(BACKWARD);
}

void leftMotorsReverse() {
    motorFL.run(BACKWARD);
    motorBL.run(FORWARD);
}

// Direction control functions for right motors
void rightMotorsForward() {
    motorFR.run(FORWARD);
    motorBR.run(BACKWARD);
}

void rightMotorsReverse() {
    motorFR.run(BACKWARD);
    motorBR.run(FORWARD);
}

// Movement functions
void forward(float dist, float speed) {
    // if (dist > 0) {
    //     deltaDist = dist;
    // } else {
    //     deltaDist = 9999999; // Move indefinitely
    // }
    // newDist = forwardDist + deltaDist;
    lastMoveDist = millis();
    dir = (TDirection)FORWARD;
    setMotorSpeed(speed);
    leftMotorsForward();
    rightMotorsForward();
}

```

```

void backward(float dist, float speed) {
    // if (dist > 0) {
    //     deltaDist = dist;
    // } else {
    //     deltaDist = 9999999; // Move indefinitely
    // }
    // newDist = reverseDist + deltaDist;
    lastMoveDist = millis();
    dir = (TDirection)BACKWARD;
    setMotorSpeed(speed);
    leftMotorsReverse();
    rightMotorsReverse();
}

void left(float ang, float speed) {
    // if (ang == 0) {
    //     deltaTicks = 99999999; // Turn indefinitely
    // } else {
    //     deltaTicks = computeDeltaTicks(ang); // Assumes this
function exists
    //     elsewhere
    // }
    // targetTicks = leftReverseTicksTurns + deltaTicks;
    lastTurnTime = millis();
    dir = (TDirection)LEFT;
    setMotorSpeed(speed);
    leftMotorsForward();
    rightMotorsReverse();
}

void right(float ang, float speed) {
    // if (ang == 0) {
    //     deltaTicks = 99999999; // Turn indefinitely
    // } else {
    //     deltaTicks = computeDeltaTicks(ang); // Assumes this
function exists

```

```

//  elsewhere
// }
// targetTicks = rightReverseTicksTurns + deltaTicks;
lastTurnTime = millis();
dir = (TDirection)RIGHT;
setMotorSpeed(speed);
leftMotorsReverse();
rightMotorsForward();
}

```

Alex.ino part of the main loop control logic code

```

if (distance > 0) {
    if ((dir == FORWARD && millis() - lastMoveDist >= distance) ||
        (dir == BACKWARD && millis() - lastMoveDist >= distance) ||
        (dir == (TDirection)STOP)) {
        // deltaDist = 0;
        // newDist = 0;
        stop();
    }
}

if (angleDur > 0) {
    if ((dir == LEFT && millis() - lastTurnTime >= angleDur) ||
        (dir == RIGHT && millis() - lastTurnTime >= angleDur) ||
        (dir == (TDirection)STOP)) {
        // deltaTicks = 0;
        // targetTicks = 0;
        stop();
    }
}
}

```

9.5. findColor() Code

```

void findColor() {
    digitalWrite(S2, LOW);
    digitalWrite(S3, LOW);
    delay(colorSensorDelay);
    redFreq = avgFREQ();
}

```

```

digitalWrite(S2, HIGH);
digitalWrite(S3, HIGH);
delay(colorSensorDelay);
greenFreq = avgFREQ();

digitalWrite(S2, LOW);
digitalWrite(S3, HIGH);
delay(colorSensorDelay);
blueFreq = avgFREQ();
}

```

9.6. avgFREQ() Code

```

int avgFREQ() {
    int total = 0;
    for (int i = 0; i < 5; i++) {
        total += pulseIn(sensorOut, digitalRead(sensorOut) == HIGH ?
LOW : HIGH);
        delay(colorAverageDelay);
    }
    return total / 5;
}

```

9.7. getColor() Code

```

void getColor() {
    // Determine color based on RGB frequencies
    // Case 1: Red
    if (redFreq < blueFreq && redFreq <= greenFreq && redFreq < 23)
    {
        detectedColor = RED;
    }
    // Case 2: Green
    else if (greenFreq < redFreq && (greenFreq - blueFreq) <= 8) {
        detectedColor = GREEN;
    }
}

```



```

    // Case 3: Unknown color - Default case if no other condition is
met
    else {
        detectedColor = UNKNOWN;
    }
}

```

9.8. Packing and Sending LiDAR Scans

```

while not ctx.isExit():
    messages = getMessages(block=True, timeout=1)
    for m in messages:
        angles, distances, qualities = PubSubMsg.getPayload(m)
        #print(f"[TLS SEND] Sending scan with {len(angles)}
points")
        data = bytearray()
        data.append(TNetType.NET_LIDAR_PACKET.value)
        for a, d, q in zip(angles, distances, qualities):
            data.extend(struct.pack("=fHh", float(a), int(d),
int(q)))
        sendNetworkData(data)

```

9.9. Receiving LiDAR Scans on PC

```

def handleLidarScan(payload: bytes):
    #print(f"[LIDAR DECODE] Received {len(payload)} bytes")
    import struct
    count = len(payload) // 8 # Each scan point is 8 bytes
    angles = []
    distances = []
    qualities = []

    for i in range(count):
        a, d, q = struct.unpack("=fHh", payload[i*8:i*8+8])
        angles.append(a)
        distances.append(d)
        qualities.append(q)

```

```

    #print(f"[LIDAR DECODE] Received {len(payload)} bytes")
    #print(f"[LIDAR DECODE] Publishing scan with {len(angles)}
points")

    publish("lidar/scan", (angles, distances, qualities))

```

9.10 getch() Code

```

char getch() {
    char buf = 0;
    struct termios old = {0};
    if (tcgetattr(0, &old) < 0)
        perror("tcsetattr()");
    old.c_lflag &= ~ICANON;
    old.c_lflag &= ~ECHO;
    old.c_cc[VMIN] = 1;
    old.c_cc[VTIME] = 0;
    if (tcsetattr(0, TCSANOW, &old) < 0)
        perror("tcsetattr ICANON");
    if (read(0, &buf, 1) < 0)
        perror("read()");
    old.c_lflag |= ICANON;
    old.c_lflag |= ECHO;
    if (tcsetattr(0, TCSADRAIN, &old) < 0)
        perror("tcsetattr ~ICANON");
    return (buf);
}

```

9.11. Fancy Alex Control Panel

Welcome and Instructions Page

```

pi@raspberrypi:~/Desktop/BareTest/RPi $ ./Alex-pi
ATTEMPTING TO CONNECT TO SERIAL. ATTEMPT # 1 of 5.
=====
WELCOME TO THE CONTROL PANEL OF THE ROBONAUTS
=====
WAITING TWO SECONDS FOR ARDUINO TO REBOOT...
DONE
=====
THE ROBONAUTS CONTROL PANEL
=====
WASD - Move    f - Stop    e - Get Stats    r - Clear Stats    q - Exit
m - Toggle Manual Mode    c - Get Color    1 - Slow Mode    2 - Normal Mode    3 - Fast Mode
o - Open Arm    p - Close Arm
=====

```

Control Page

```
=====
Get Color
=====
Currently in NORMAL MODE
=====

----- ALEX COLOR SENSOR -----
Red (R) frequency:    6
Green (G) frequency:  47
Blue (B) frequency:  48
Detected Color:       RED
-----

Command OK

=====
CLOSE Arm
=====
Currently in NORMAL MODE
=====
Command OK
=====

Open Arm
=====
Currently in NORMAL MODE
=====
Command OK
=====
```

9.12. Filtering Algorithm for Lidar Scans

```
if results and current_round["r"] > INITIAL_ROUNDS_IGNORED:
    # Define filtering criteria
    distance_threshold = 100
    quality_threshold = 15
    filtered_results = (
        [angle for angle, distance, quality in zip(results[0],
results[1], results[2]) if distance > distance_threshold and
quality > quality_threshold],
        [distance for distance, quality in zip(results[1], results[2])
if distance > distance_threshold and quality > quality_threshold],
        [quality for quality in results[2] if quality >
quality_threshold]
    )
    #print("Filtered results:", filtered_results)
    publish(LIDAR_SCAN_TOPIC, filtered_results)
```